

AI 활용 학습 · Python & Git 기초

나만의 프롬프트 관리 프로그램

Prompt Manager 개발 결과 보고서

제출자: 오주연

GitHub: https://github.com/jjjoy1114/A1_1_prompt_manager

작성일: 2026년 8월 7일

1. 프로젝트 개요

이 프로젝트는 여러 미션을 수행하며 흩어진 AI 프롬프트를 한곳에서 체계적으로 저장·검색·관리할 수 있는 파이썬(Python) 콘솔 프로그램(Prompt Manager)을 개발하고, Git과 GitHub을 활용한 버전 관리 전 과정을 실습한 결과물이다. 프로그램은 터미널에서 메뉴 번호를 입력해 기능을 선택하는 방식으로 동작하며, 외부 라이브러리 없이 파이썬 기본 문법만으로 구현하였다.

기본 데이터로는 이전 미션인 B1-2(멀티모달 스포츠 영상 제작)에서 실제로 작성한 프롬프트 3개를 등록하였다. 보너스 과제 1(JSON 저장·불러오기 및 카테고리별 Markdown 내보내기)까지 추가로 구현하여, 프로그램을 종료한 뒤에도 데이터가 유지된다.

항목	내용
프로젝트명	prompt-manager
개발 언어	Python 3.13.15 (과제 기준 3.10 이상 충족)
개발 도구	Visual Studio Code, Git 2.55.0.windows.3
GitHub 저장소	https://github.com/jjjoy1114/A1_1_prompt_manager
총 커밋 수	11개 (요구사항 10개 이상 충족)
브랜치 작업	feature/list-prompts, docs/author-info (생성 및 병합 완료)
구현 범위	필수 기능 전체 + 보너스1(JSON 영속화·Markdown 내보내기)

2. 과제 요구사항 충족 여부

평가 영역	상태	확인 결과
콘솔 프로그램	충족	번호 선택형 메뉴, 기능 수행 후 메뉴 복귀, 종료 처리
잘못된 입력 처리	충족	메뉴에 없는 번호 입력 시 안내 후 메뉴 재표시
기본 프롬프트	충족	이전 미션(B1-2) 프롬프트 3개 등록
필수 기능	충족	추가·목록·카테고리·검색·상세·즐거찾기 구현
데이터 구조	충족	리스트 안에 딕셔너리 사용
코드 구조	충족	기능별 함수 분리
커밋 10개 이상	충족	의미 있는 커밋 11개
브랜치·병합	충족	브랜치 2개 생성 후 main에 병합(--no-ff)

Git 명령어	충족	init·add·commit·push·pull·checkout·clone·merge 사용
.gitignore / README	충족	불필요 파일 제외, README 작성
GitHub 업로드	충족	공개 저장소 main 브랜치 업로드
보너스1	완료	JSON 저장·불러오기 + 카테고리별 Markdown 내보내기

3. 개발 환경 설정

VSCode에 Python 확장을 설치하고, 터미널에서 Python과 Git 버전을 확인한 뒤 Git 사용자 정보와 기본 브랜치(main)를 설정하였다. 아래 그림과 같이 Python 3.13.15, Git 2.55.0.windows.3, 사용자 이름 (jjjoy1114)·이메일 및 기본 브랜치(main)를 확인하였다.

```

PS C:\prompt-manager> & C:\Users\USER\AppData\Local\Programs\Python\Python313\python.exe c:/prompt-manager/hello.py
Hello
PS C:\prompt-manager> python --version
Python 3.13.15
PS C:\prompt-manager> git --version
git version 2.55.0.windows.3
PS C:\prompt-manager> git config --global user.name
jjjoy1114
PS C:\prompt-manager> git config --global user.email
sadykr86@gmail.com
PS C:\prompt-manager> git config --global init.defaultBranch
main
PS C:\prompt-manager>

```

그림 1. Python·Git 버전 확인 및 Git 사용자 설정

이어서 hello.py에 print("Hello") 코드를 작성하고 실행하여, 파이썬 파일 생성과 실행 환경이 정상 동작함을 확인하였다.

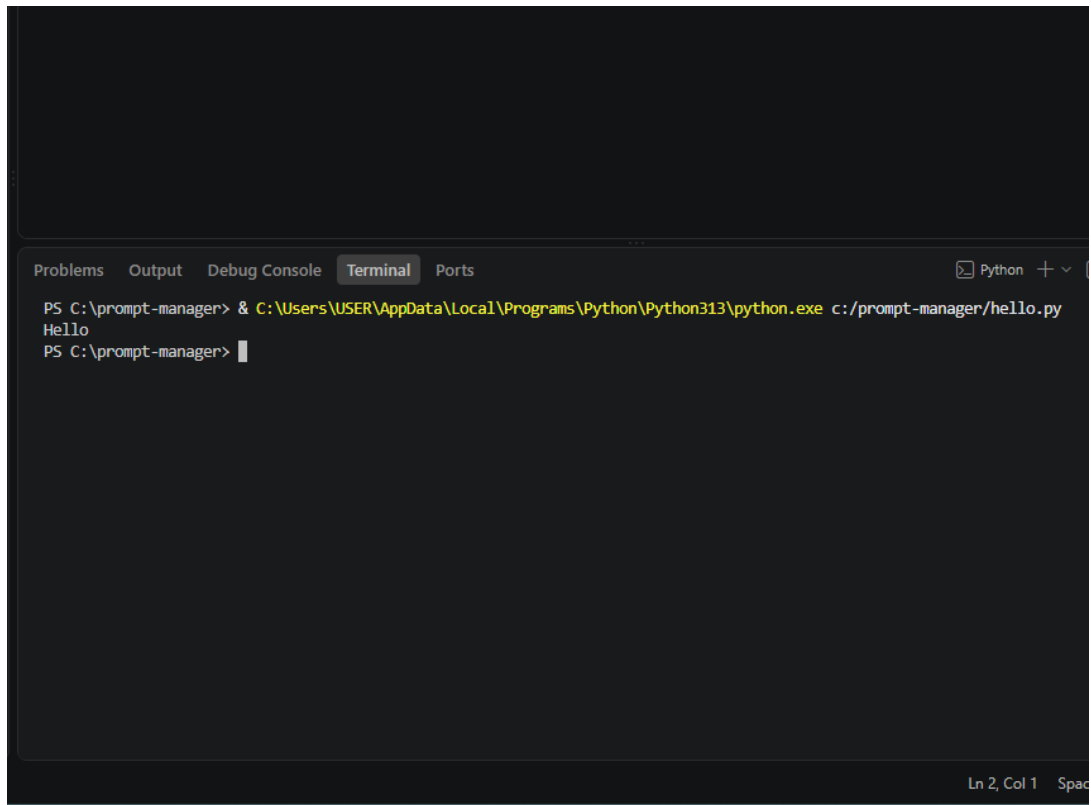


그림 2. `hello.py` 실행 → `Hello` 출력

4. 프로그램 구성

4.1 데이터 구조 (리스트 + 딕셔너리)

프로그램은 `prompts`라는 리스트 안에 여러 개의 딕셔너리를 넣는 방식으로 프롬프트를 관리한다. 딕셔너리 하나가 프롬프트 하나를 뜻하며, 각 딕셔너리는 다음 네 개의 키로 구성된다.

키 이름	저장 내용	값의 종류
title	프롬프트 제목	문자열
content	프롬프트 전체 내용	문자열
category	프롬프트 분류	문자열
favorite	즐거찾기 여부	참·거짓(True/False)

기본 카테고리는 텍스트 생성, 이미지 생성, 영상 생성, 페르소나, 자동화, 기타이다.

4.2 리스트와 딕셔너리 비교 및 선택 근거

두 자료구조는 역할이 다르므로 장단점을 비교한 뒤 함께 사용하였다.

구분	리스트	딕셔너리
----	-----	------

역할	여러 프롬프트를 순서대로 저장	프롬프트 1개의 정보를 이름(키)별로 저장
장점	번호를 붙여 순서대로 출력·접근하기 쉬움	제목·내용·카테고리를 키로 명확히 구분
단점	데이터가 많아지면 전체 순회 검색이 느려짐	키 이름을 정확히 사용해야 함
사용 예	<code>prompts[0]</code>	<code>prompt["title"]</code>
선택 이유	전체 목록 관리에 적합	프롬프트 한 건의 속성 관리에 적합

현재처럼 프롬프트가 수 개 수준인 작은 목록은 처음부터 차례로 확인해도 부담이 작지만, 데이터가 수천 개로 늘어나면 매번 전체 리스트를 비교해야 하므로 검색이 느려질 수 있다. 본 프로젝트는 학습용이며 데이터 규모가 작아 초보자가 이해하기 쉬운 리스트·딕셔너리 조합을 선택하였고, 별도의 색인 구조나 데이터베이스는 도입하지 않았다.

4.3 기능별 함수 구성 및 분리 이유

모든 로직을 하나의 함수에 몰아넣지 않고 역할 단위로 함수를 분리하였다. 기능별로 나누면 각 기능을 독립적으로 이해·수정·재사용할 수 있고, 메뉴 번호와 함수가 1:1로 대응되어 오류가 생겼을 때 원인을 찾기 쉽다. 또한 `input_required()`나 `get_index_by_number()`처럼 공통으로 쓰이는 입력 처리는 한 곳에 모아 중복 코드를 줄였다.

함수	담당 기능
<code>show_menu()</code>	메인 메뉴 출력
<code>input_required()</code>	빈 입력 방지 (다시 입력 요청)
<code>select_category()</code>	카테고리 번호 선택 또는 직접 입력
<code>is_duplicate_title()</code>	같은 제목 중복 여부 확인
<code>add_prompt()</code>	새 프롬프트 등록
<code>show_prompt_list()</code>	전체 목록 출력
<code>show_by_category()</code>	카테고리별 조회
<code>search_prompt()</code>	제목·내용 키워드 검색
<code>show_detail()</code>	프롬프트 상세 내용 출력
<code>toggle_favorite()</code>	즐거찾기 추가·해제
<code>show_favorites()</code>	즐거찾기 목록 출력
<code>save_prompts() / load_prompts()</code>	JSON 저장·불러오기 (보너스1)
<code>export_markdown()</code>	카테고리별 Markdown 내보내기 (보너스1)

main()	메뉴 반복 및 입력에 따른 함수 호출
--------	----------------------

5. 필수 기능 구현 및 실행 결과

프로그램을 실행하면 1~8번 기능과 0번 종료 메뉴가 표시되고, 번호를 입력하면 해당 기능이 실행된 뒤 다시 메인 메뉴로 돌아온다.

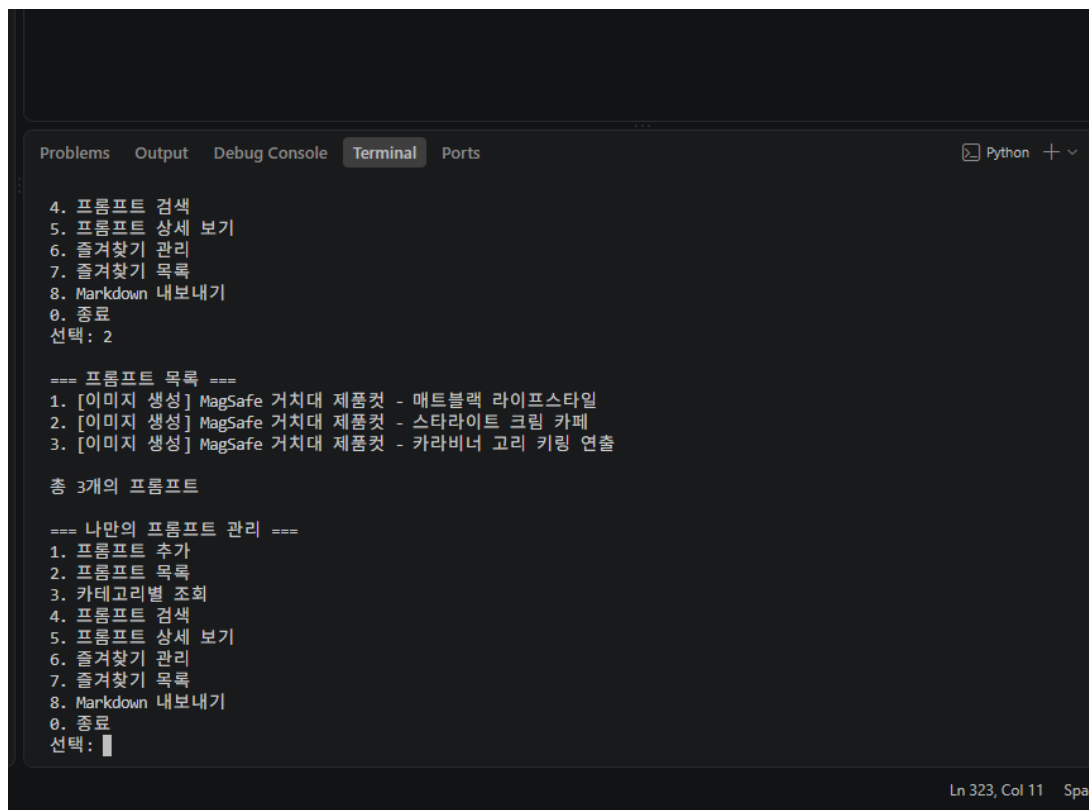


그림 3. 번호 선택형 메인 메뉴 실행 화면

메뉴에 없는 번호를 입력하면 안내 문구를 표시하고 메뉴를 다시 보여준다. 9를 입력하면 ‘잘못된 번호입니다. 다시 입력해주세요.’를 출력한 뒤 메뉴를 재표시한다.

```
Problems Output Debug Console Terminal Ports Python + v

환영합니다! 나만의 프롬프트 관리 프로그램입니다.

=== 나만의 프롬프트 관리 (총 4개) ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
8. Markdown 내보내기
0. 종료
선택: 9
잘못된 번호입니다. 다시 입력해주세요.

=== 나만의 프롬프트 관리 (총 4개) ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
8. Markdown 내보내기
0. 종료
선택: █
```

그림4. 잘못된 메뉴 번호(9) 입력 시 안내 후 메뉴 재표시

제목·내용·카테고리를 입력하면 새 프롬프트가 등록된다. 빈 값은 저장할 수 없고, 같은 제목이 이미 있으면 다른 제목을 요구한다. 실제로 B1-2에서 작성한 프롬프트('s1 아웃도어무드(배낭 결합 컷)')를 추가하여 총 4개로 늘어난 것을 확인하였다.

```
0. 종료
선택:
잘못된 번호입니다. 다시 입력해주세요.

=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
8. Markdown 내보내기
0. 종료
선택: 2

=== 프롬프트 목록 ===
1. [이미지 생성] MagSafe 거치대 제품컷 - 매트블랙 라이프스타일
2. [이미지 생성] MagSafe 거치대 제품컷 - 스타라이트 크림 카페
3. [이미지 생성] MagSafe 거치대 제품컷 - 카라비너 고리 키링 연출
4. [이미지 생성] s1 아웃도어무드(배낭 결합 컷)

총 4개의 프롬프트

=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
8. Markdown 내보내기
0. 종료
선택: █
```

Ln 323, Col 11 Spa

그림5. 프롬프트 추가 후 전체 목록 (총 4개)

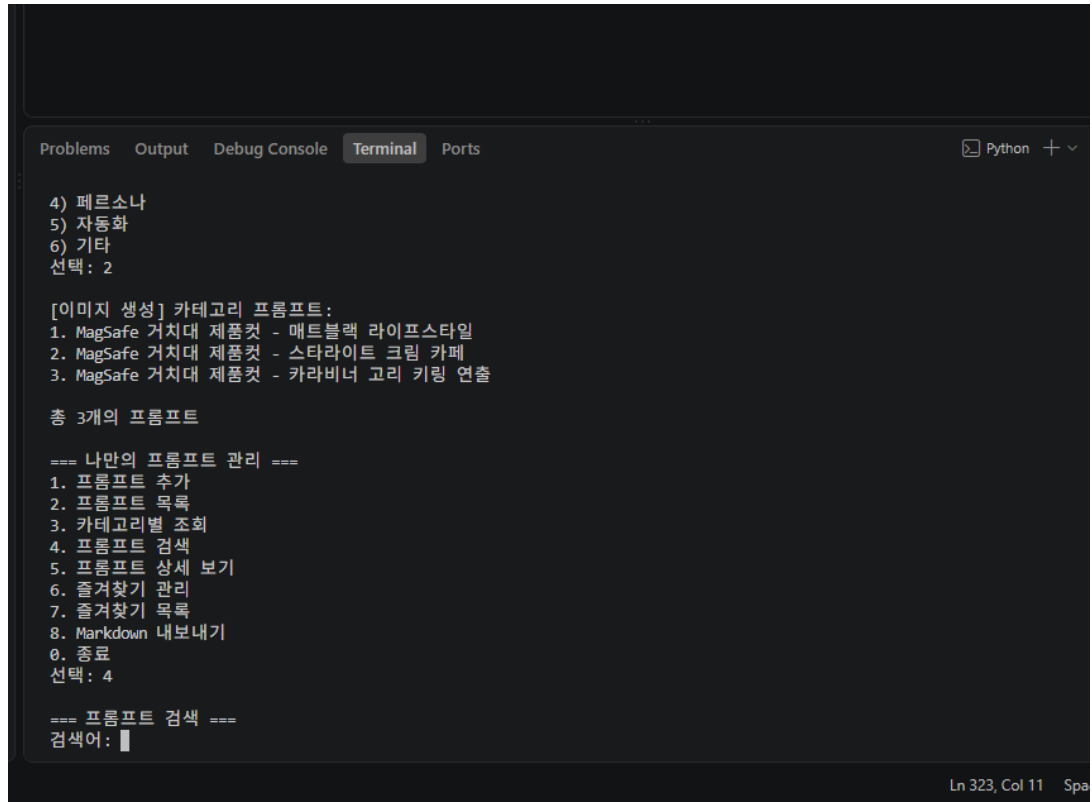


그림6. 카테고리별 조회 (이미지 생성 카테고리)

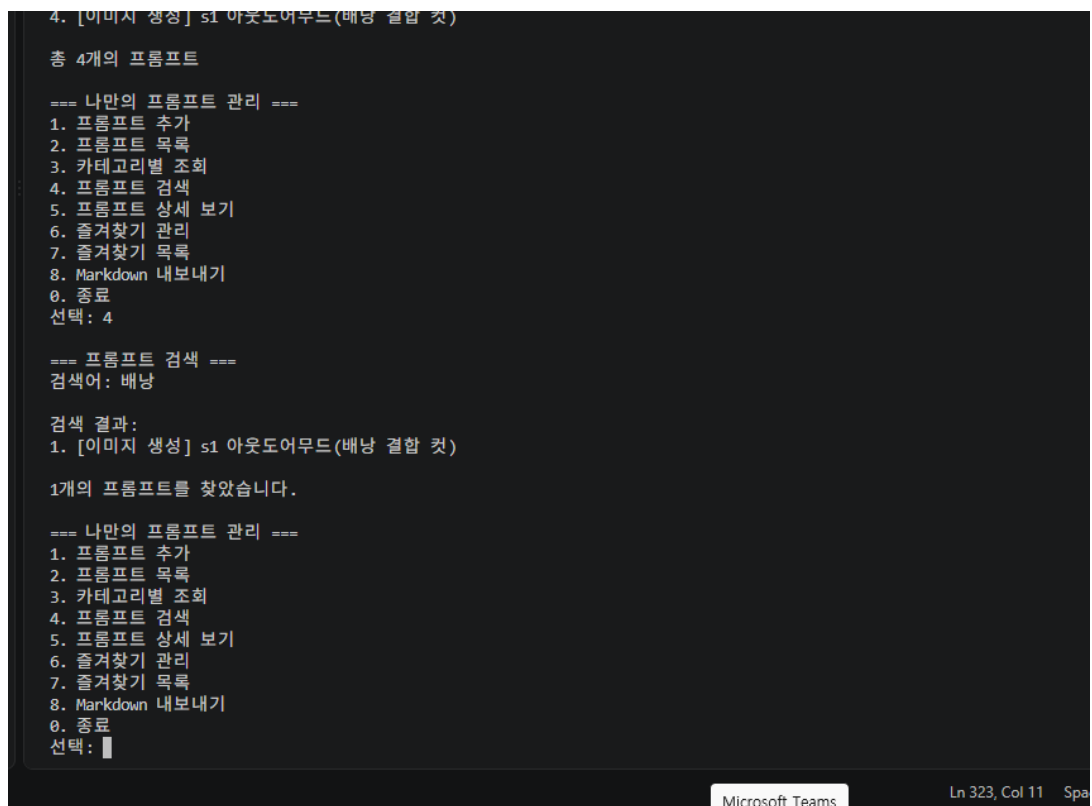


그림7. 제목 내용 키워드 검색 결과 (검색어: 배낭)


```
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
8. Markdown 내보내기
0. 종료
선택: 5

=== 프롬프트 상세 보기 ===
번호 입력: 4

제목: s1 아웃도어무드 (배낭 결함 컷)
카테고리: 이미지 생성
즐거찾기: 없음

내용:
첨부한 이미지 속 제품 원형 변경 절대 금지, 포인트 색상 등 색상변경 가능 A close-up product shot of a sleek matte black
color with a bright neon orange point, securely clipped to the webbing of a rugged tactical hiking backpack. Water droplets on the
al, dark misty pine forest background, gorpcore aesthetic, high-end outdoor gear photography, cinematic lighting, 8k,
ar 9:16

=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
8. Markdown 내보내기
0. 종료
선택: █
```

Ln 323, Col 11 Spa

그림 8. 프롬프트 상세 보기 (제목·카테고리·즐거찾기·전체 내용)

```
0. 종료
선택: 6

=== 즐겨찾기 관리 ===
프롬프트 번호 입력: 1
'MagSafe 거치대 제품컷 - 매트블랙 라이프스타일' 프롬프트를 즐겨찾기에 추가했습니다!

=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
8. Markdown 내보내기
0. 종료
선택: 7

=== 즐겨찾기 목록 ===
1. [이미지 생성] MagSafe 거치대 제품컷 - 매트블랙 라이프스타일 ★

총 1개의 즐겨찾기

=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
8. Markdown 내보내기
0. 종료
선택: █
```

Ln 323, Col 11 Spa

그림 9. 즐겨찾기 추가 후 즐겨찾기 목록 (★ 표시)

5.1 입력 검증 정책

- 빈 값 방지: `input_required()`에서 공백만 입력하면 ‘입력값이 비어있습니다. 다시 입력해주세요.’를 출력하고 다시 입력받는다.
- 숫자·범위 검증: 프롬프트 번호는 `isdigit()`로 숫자 여부를 확인하고, 1~전체 개수 범위를 벗어나면 ‘잘못된 번호입니다.’로 처리한다.
- 메뉴 번호: 0~8 이외의 값은 안내 후 메뉴를 재표시하여 프로그램이 종료되지 않도록 한다.
- 길이·특수문자: 제목·내용에는 최대 길이나 허용 문자 제한을 두지 않았다(자유 입력). 빈 값과 중복 제목만 차단하며, 실제 서비스라면 길이 제한과 이스케이프 처리가 추가로 필요하다.

5.2 중복 제목 처리 정책

`is_duplicate_title()`이 `lower()`를 사용해 영문 대소문자를 무시하고 제목 중복을 검사한다. 같은 제목이 있으면 저장을 차단하고 다른 제목을 요청한다. 덮어쓰기나 자동 번호 부여(제목(2) 등) 방식은 채택하지 않았는데, 제목을 프롬프트를 구분하는 고유 식별자처럼 다루어 검색·상세 보기에서 혼동이 없도록 하기 위함이다.

5.3 검색 동작 방식

`search_prompt()`는 검색어를 `lower()`로 소문자화한 뒤 각 프롬프트의 제목과 내용에 포함되는지 검사하는 부분 일치 방식이다. 따라서 영문은 대소문자를 구분하지 않고(MagSafe = magsafe), 제목·내용의 일부만 입력해도 검색된다(예: ‘배낭’ → ‘s1 아웃도어무드(배낭 결합 컷)’). 토큰화, 정규식, 불용어 제거는 사용하지 않았으며 띄어쓰기나 오타는 자동 보정하지 않는다.

5.4 카테고리 운영 방식

- 직접 입력: 카테고리 선택 시 번호(1~6) 대신 문자열을 입력하거나 ‘기타’를 선택하면 입력한 이름을 그대로 카테고리로 저장한다.
- 빈 카테고리 조회: 해당 카테고리에 프롬프트가 없으면 ‘[카테고리] 카테고리의 프롬프트가 없습니다.’를 출력하고 메뉴로 돌아온다.
- 변경 시 수정 위치: 카테고리를 추가·삭제·이름 변경하려면 `prompt_manager.py` 상단의 `CATEGORIES` 리스트를 수정한다. 이름을 바꾼 경우 기존 `prompts.json`의 `category` 값은 자동 변환되지 않으므로 파일에서 직접 확인·수정해야 한다.

6. 보너스 과제 1 구현

6.1 JSON 저장 및 불러오기

파이썬 기본 json 기능으로 프롬프트를 prompts.json 파일에 저장한다. 프로그램 시작 시 load_prompts()가 저장 내용을 읽고, 프롬프트 추가·즐거찾기 변경 시 save_prompts()가 즉시 저장하므로, 종료 후 다시 실행해도 데이터와 즐거찾기 상태가 그대로 유지된다. UTF-8과 ensure_ascii=False로 한글이 깨지지 않게 저장하였다.

```
선택: 0
프로그램을 종료합니다.
PS C:\prompt-manager> python prompt_manager.py

=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
8. Markdown 내보내기
0. 종료
선택: 2

=== 프롬프트 목록 ===
1. [이미지 생성] MagSafe 거치대 제품컷 - 매트블랙 라이프스타일
2. [이미지 생성] MagSafe 거치대 제품컷 - 스타라이트 크림 카페
3. [이미지 생성] MagSafe 거치대 제품컷 - 카라비너 고리 키링 연출
4. [이미지 생성] s1 아웃도어무드(배낭 결합 컷) ★

총 4개의 프롬프트

=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
8. Markdown 내보내기
0. 종료
선택: █
```

그림10. 종료 후 재실행 시 프롬프트가 그대로 유지되는 화면

0번으로 종료할 때 따로 저장 명령을 실행하지는 않는다. 프롬프트를 추가하거나 즐겨찾기를 바꾸는 시점에 이미 JSON 파일에 즉시 저장되므로, 종료 후에도 내용이 그대로 남는다.

6.1.1 저장 형식 선택 근거 (JSON vs CSV vs DB)

구분	JSON	CSV	DB(SQLite 등)
저장 구조	리스트·딕셔너리를 그대로 저장	행과 열로 저장	테이블 스키마
여러 줄 내용	긴 프롬프트도 관리 쉬움	줄바꿈이 있으면 불편	가능하나 설정 필요
True/False	불리언 자료형 그대로	문자열처럼 보일 수 있음	지원
도입 비용	표준 라이브러리로 즉시 사용	간단하나 구조 표현 제한	스키마·연결 필요(과함)
선택 결과	채택	미채택	미채택

제목·내용·카테고리·즐거찾기라는 중첩 구조를 외부 라이브러리 없이 그대로 저장하기에 JSON이 가장 적합하여 선택하였다. CSV는 여러 줄 프롬프트와 불리언 처리가 불편하고, 데이터베이스는 이 정도 규모에는 과도한 선택이다.

6.2 카테고리별 Markdown 내보내기

8번 메뉴를 실행하면 프롬프트를 카테고리별로 묶어 exports 폴더에 Markdown(.md) 파일로 저장한다. 각 파일에는 제목, 즐겨찾기 표시, 내용 전체가 기록된다.

```
0. 종료
선택: 7

=== 즐겨찾기 목록 ===
1. [이미지 생성] s1 아웃도어무드(배낭 결합 컷) ★

총 1개의 즐겨찾기

=== 나의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
8. Markdown 내보내기
0. 종료
선택: 8

=== Markdown 내보내기 ===
Markdown 파일을 exports/ 폴더에 저장했습니다!
생성된 카테고리 파일 수: 1개

=== 나의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
8. Markdown 내보내기
0. 종료
선택: []
```

그림11. Markdown 내보내기 실행 결과(exports 폴더 생성)

7. Git 작업 및 브랜치 병합

git init으로 저장소를 시작하고 기능·문서 단위로 총 11개의 커밋을 남겼다. 프롬프트 목록 기능은 feature/list-prompts 브랜치에서, 작성자·문서 정보는 docs/author-info 브랜치에서 작업한 뒤 각각 main에 병합(--no-ff)하였다.

7.1 커밋 단위와 메시지 정책

- 커밋 단위: 한 커밋에 한 가지 변경 목적만 담기도록 기능·문서·데이터 등 의미 있는 최소 단위로 나눈다.

- 메시지 규칙: Conventional Commits 스타일 접두사를 사용한다. feat(기능 추가), data(데이터 추가), docs(문서), chore(설정·정리), merge(병합).
- 예시: 'feat: add prompt manager with menu and core features', 'data: add base prompts from B1-2 mission', 'chore: ignore generated exports and test file', 'merge: list-prompts feature into main'.

7.2 브랜치 전략 (분리 기준과 병합 타이밍)

- 분리 기준: 기능 개발은 feature/*, 문서 작업은 docs/* 브랜치로 나눈다. 서로 다른 성격의 작업이 main에서 뒤섞이지 않게 하기 위함이다.
- 병합 타이밍: 해당 브랜치에서 기능이 정상 동작함을 실행으로 확인한 뒤에만 main으로 병합한다.
- 병합 방식: git merge <브랜치> --no-ff 옵션으로 병합 커밋을 생성하여, 로그 그래프에 분기·병합 지점이 명확히 남도록 한다.

7.3 병합 충돌 처리 절차

같은 파일의 동일한 부분을 서로 다르게 수정하면 충돌이 발생한다. 다음 순서로 처리한다.

- 1. 원인 확인: git status로 충돌 파일을 확인하고, 파일 내부의 <<<<<<, =====, >>>>>> 표시 구간을 찾는다.
- 2. 해결: 남길 내용을 정리하고 충돌 표시 기호를 삭제하여 하나의 내용으로 합친다.
- 3. 검증: 프로그램을 다시 실행하여 기능이 정상 동작하는지 확인한다.
- 4. 기록: git add로 해결한 파일을 스테이징하고 git commit으로 해결 내용을 저장한다.

7.4 사용한 Git 명령어

명령어	수행 내용
git init	로컬 저장소 초기화
git add / commit	기능 단위 변경 이력 기록
git checkout -b	feature/list-prompts, docs/author-info 브랜치 생성·이동
git merge --no-ff	기능 브랜치를 main에 병합 (병합 커밋 생성)
git push	main 브랜치를 GitHub에 업로드
git pull	GitHub에서 수정한 README를 로컬로 반영
git clone	공개 저장소(octocat/Hello-World) 복제 실행

```
Problems Output Debug Console Terminal Ports
Python + v

* 78133c2 data: add base prompts from B1-2 mission
* 7a3ba10 feat: add prompt manager with menu and core features
* 672142b chore: initialize project with README and gitignore
PS C:\prompt-manager> git add .gitignore
>> git commit -m "chore: ignore generated exports and test file"
[main 463ba07] chore: ignore generated exports and test file
1 file changed, 2 insertions(+)
PS C:\prompt-manager> git add prompt_manager.py
>> git commit -m "feat: show total prompt count in menu header"
[main ac38bdc] feat: show total prompt count in menu header
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\prompt-manager> git add README.md
>> git commit -m "docs: add file structure section"
>> git log --oneline
[main 21415e1] docs: add file structure section
1 file changed, 6 insertions(+)
21415e1 (HEAD -> main) docs: add file structure section
ac38bdc feat: show total prompt count in menu header
463ba07 chore: ignore generated exports and test file
5ce3668 merge: complete README into main
0090e2d (docs/author-info) docs: complete README with usage, features, and author info
87af408 merge: list-prompts feature into main
3e7558b (feature/list-prompts) feat: add welcome message on program start
78133c2 data: add base prompts from B1-2 mission
7a3ba10 feat: add prompt manager with menu and core features
672142b chore: initialize project with README and gitignore
PS C:\prompt-manager>
```

그림 12. git log --oneline — 의미 있는 커밋 10 개

```
15
16 DEFAULT_PROMPTS = [
17     {
18         "title": "MagSafe 거치대 제품컷 - 매트블랙 라이프스타일",
19         "content": "A sleek matte black metallic MagSafe phone mount, set up as a minimal stand on a
20         "category": "이미지 생성",
21         "favorite": False,
22     },
23     {
24         "title": "MagSafe 거치대 제품컷 - 스타라이트 크림 카페",
25         "content": "A sleek, minimalist MagSafe smartphone mount in a soft starlight cream color. set
26
Problems Output Debug Console Terminal Ports
Python + v

PS C:\prompt-manager> git checkout main
>> git merge feature/list-prompts --no-ff -m "merge: list-prompts feature into main"
Switched to branch 'main'
Merge made by the 'ort' strategy.
prompt_manager.py | 82 ++++++++-----
1 file changed, 14 insertions(+), 68 deletions(-)
PS C:\prompt-manager> git checkout main
>> git merge feature/list-prompts --no-ff -m "merge: list-prompts feature into main"
>> git log --oneline --graph --all
Already on 'main'
Already up to date.
* 87af408 (HEAD -> main) merge: list-prompts feature into main
|\
| * 3e7558b (feature/list-prompts) feat: add welcome message on program start
|/
* 78133c2 data: add base prompts from B1-2 mission
* 7a3ba10 feat: add prompt manager with menu and core features
* 672142b chore: initialize project with README and gitignore
PS C:\prompt-manager>
```

그림 12-2. git log --graph --all — 브랜치 분기·병합선

```

=== 나만의 프롬프트 관리 ===
1. 프롬프트 추가
2. 프롬프트 목록
3. 카테고리별 조회
4. 프롬프트 검색
5. 프롬프트 상세 보기
6. 즐겨찾기 관리
7. 즐겨찾기 목록
8. Markdown 내보내기
0. 종료
선택: 0
프로그램을 종료합니다.
PS C:\prompt-manager> ^C
PS C:\prompt-manager> git add .gitignore README.md
>> git commit -m "chore: initialize project with README and gitignore"
[main (root-commit) 672142b] chore: initialize project with README and gitignore
2 files changed, 14 insertions(+)
create mode 100644 .gitignore
create mode 100644 README.md
PS C:\prompt-manager> git add prompt_manager.py
>> git commit -m "feat: add prompt manager with menu and core features"
[main 7a3ba10] feat: add prompt manager with menu and core features
1 file changed, 323 insertions(+)
create mode 100644 prompt_manager.py
PS C:\prompt-manager> git add prompts.json
>> git commit -m "data: add base prompts from B1-2 mission"
[main 78133c2] data: add base prompts from B1-2 mission
1 file changed, 26 insertions(+)
create mode 100644 prompts.json
PS C:\prompt-manager> git log --oneline
78133c2 (HEAD -> main) data: add base prompts from B1-2 mission
7a3ba10 feat: add prompt manager with menu and core features
672142b chore: initialize project with README and gitignore
PS C:\prompt-manager>

```

jjjoy1114 (now) Ln 323, Col 11 Spa

[참고] 최초 커밋 3개(chore·feat·data)

```

elif choice == "4":
    search_prompt()
elif choice == "5":
    show_detail()
elif choice == "6":
    toggle_favorite()
elif choice == "7":
    show_favorites()
elif choice == "8":
    export_markdown()
elif choice == "0":
    print("프로그램을 종료합니다.")
    break
else:
    print("잘못된 번호입니다. 다시 입력해주세요.")

```

Problems 2 Output Debug Console Terminal Ports Python + v

```

672142b chore: initialize project with README and gitignore
PS C:\prompt-manager> git checkout -b feature/list-prompts
Switched to a new branch 'feature/list-prompts'
PS C:\prompt-manager>

```

Ln 325, Col 16 Spa

[참고] feature/list-prompts 브랜치 생성

```
Problems Output Debug Console Terminal Ports Python + v

To https://github.com/jjjoy1114/A1_1_prompt_manager.git
* [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\prompt-manager> cd ..
>> git clone https://github.com/octocat/Hello-World.git
>> cd Hello-World
>> dir
>> git log --oneline
>> cd ../prompt-manager
Cloning into 'Hello-World'...
remote: Enumerating objects: 13, done.
remote: Total 13 (delta 0), reused 0 (delta 0), pack-reused 13 (from 1)
Receiving objects: 100% (13/13), done.

디렉터리: C:\Hello-World

Mode                LastWriteTime         Length Name
----                -
-a-----      2026-08-07 오전 11:17             14 README
7fd1a60 (HEAD -> master, origin/master, origin/HEAD) Merge pull request #6 from Spaceghost/patch-1
7629413 New line at end of file. --Signed off by Spaceghost
553c207 first commit

PS C:\prompt-manager> 
```

그림 13. git clone 실습(octocat/Hello-World)

```
Problems Output Debug Console Terminal Ports Python + v

Receiving objects: 100% (13/13), done.

디렉터리: C:\Hello-World

Mode                LastWriteTime         Length Name
----                -
-a-----      2026-08-07 오전 11:17             14 README
7fd1a60 (HEAD -> master, origin/master, origin/HEAD) Merge pull request #6 from Spaceghost/patch-1
7629413 New line at end of file. --Signed off by Spaceghost
553c207 first commit

PS C:\prompt-manager> git pull
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 1013 bytes | 53.00 KiB/s, done.
From https://github.com/jjjoy1114/A1_1_prompt_manager
  21415e1..f51316d  main      -> origin/main
Updating 21415e1..f51316d
Fast-forward
 README.md | 26 ++++++-----
 1 file changed, 8 insertions(+), 18 deletions(-)
PS C:\prompt-manager> 
```

그림 14. git pull 실행 결과(Fast-forward)


```
Problems Output Debug Console Terminal Ports
Python + v

>> git commit -m "docs: add file structure section"
>> git log --oneline
[main 21415e1] docs: add file structure section
1 file changed, 6 insertions(+)
21415e1 (HEAD -> main) docs: add file structure section
ac38bdc feat: show total prompt count in menu header
463ba07 chore: ignore generated exports and test file
5ce3668 merge: complete README into main
0090e2d (docs/author-info) docs: complete README with usage, features, and author info
87af408 merge: list-prompts feature into main
3e7558b (feature/list-prompts) feat: add welcome message on program start
78133c2 data: add base prompts from B1-2 mission
7a3ba10 feat: add prompt manager with menu and core features
672142b chore: initialize project with README and gitignore
PS C:\prompt-manager> git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 27, done.
Counting objects: 100% (27/27), done.
Delta compression using up to 8 threads
Compressing objects: 100% (25/25), done.
Writing objects: 100% (27/27), 7.39 KiB | 756.00 KiB/s, done.
Total 27 (delta 13), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (13/13), done.
To https://github.com/jjjoy1114/A1_1_prompt_manager.git
 * [new branch]    main -> main
branch 'main' set up to track 'origin/main'.
PS C:\prompt-manager> |
```

[참고] `git push -u origin main` 성공

8. GitHub 저장소 업로드 결과

최종 코드는 공개 GitHub 저장소의 main 브랜치에 업로드하였다. 주요 파일은 실행 코드

`prompt_manager.py`, 설명 문서 `README.md`, 저장 데이터 `prompts.json`, 제외 규칙 `.gitignore`이며, 자동 생성되는 `exports` 폴더와 `__pycache__`는 `.gitignore`로 제외하였다.

저장소 URL: https://github.com/jjjoy1114/A1_1_prompt_manager

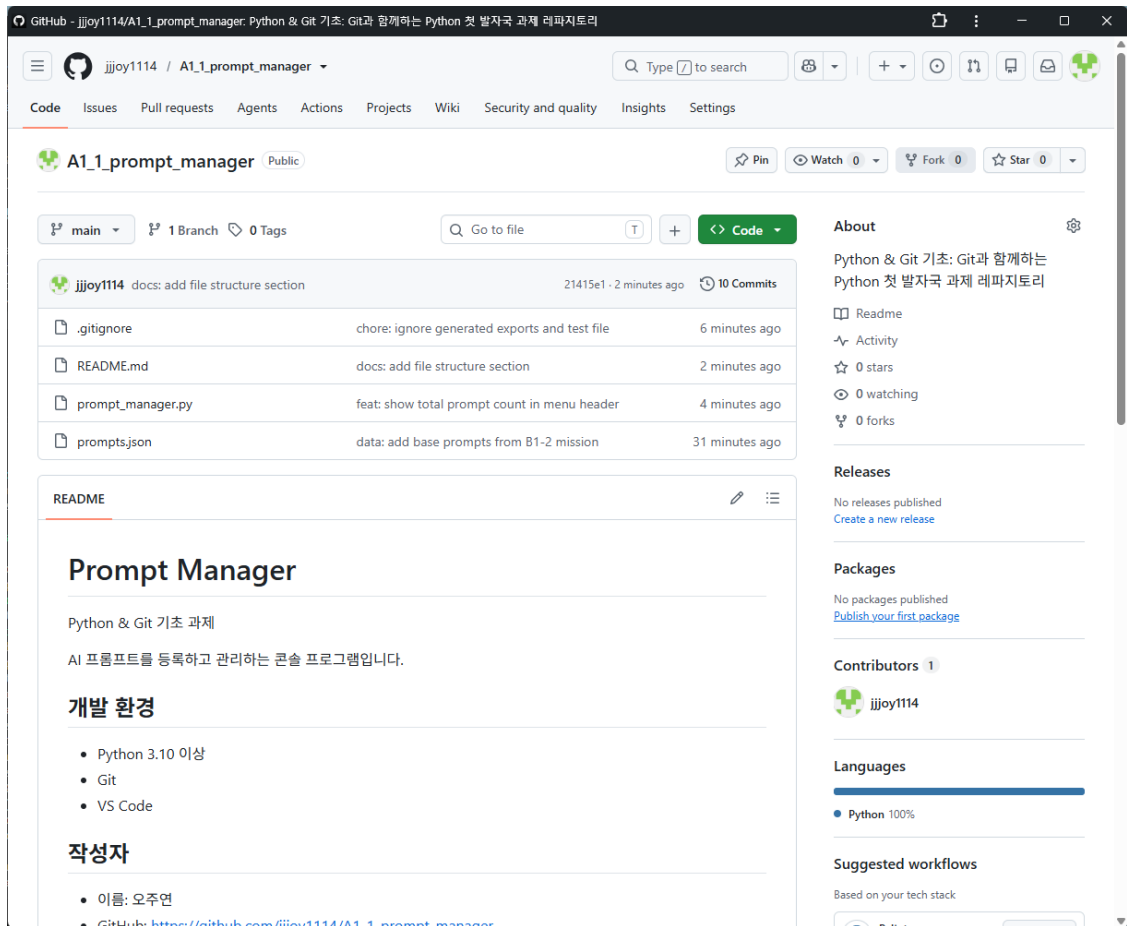
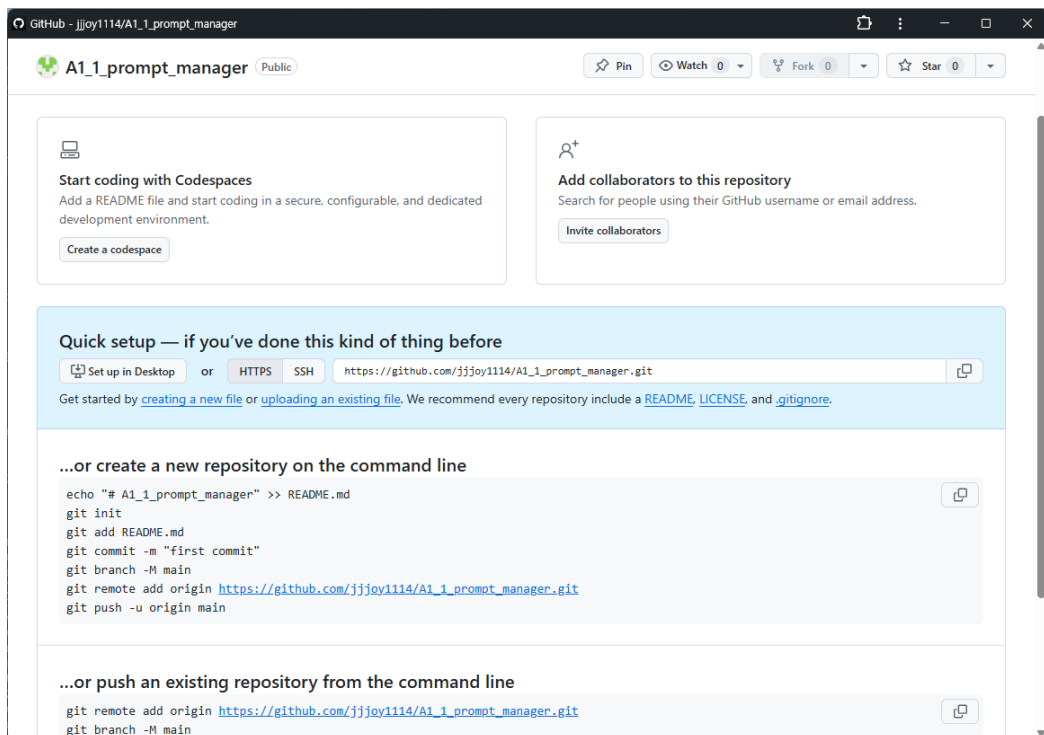


그림 15. GitHub 저장소 웹 화면 (파일 구성 커밋 10개 README 렌더링)



[참고] GitHub 저장소 생성 직후 화면

9. 학습 성과 자가 점검

- VSCode에서 Python 파일을 생성하고 실행할 수 있다.
- 터미널에서 Python/Git 버전을 확인하고 필요한 설정을 점검할 수 있다.
- 파이썬 기초 문법(변수, 조건문, 반복문, 함수, 리스트/딕셔너리)을 설명할 수 있다.
- Git이 무엇이고 왜 필요한지 설명할 수 있다.
- init, add, commit, push, pull, checkout, clone, merge가 각각 무엇을 하는지 설명할 수 있다.
- 브랜치를 생성하고 병합(--no-ff)할 수 있다.
- GitHub에 코드를 업로드하고 관리할 수 있다.